UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS

BODEGACONTROL

TALLER 3 · API REST E INTEGRACIÓN
DOCUMENTO TÉCNICO

Curso: SI806V – Desarrollo Adaptativo e Integración de Software

Profesor: Ramos Montes, Carlos Nelson

INTEGRANTES

Aguirre Alvarado, Giancarlo

Dionicio Achachagua, Cesar Alonso

Salazar Ramos, Ernesto Ramón

Ciclo: 2026-2

Lima – Perú

Índice

1. Problema....2
2. Arquitectura....2
3. Modelo de datos....2
4. Endpoints....3
5. Reglas....4
6. Pruebas....4

1. Problema

En una bodega de barrio las ventas se anotan a mano o no se anotan. El dueño no sabe cuánto stock real tiene, qué productos se pierden por vencimiento ni cuánto gana. Cuando delega la caja, pierde el control de lo vendido.

El usuario es el dueño de la bodega (rol **PROPIETARIO**) y el cajero de turno (rol **CAJERO**). BodegaControl registra cada venta desde el celular, descuenta el stock en el mismo momento y genera reportes por periodo. El cajero vende; el propietario además gestiona el catálogo, el inventario y los reportes.

2. Arquitectura

La solución tiene tres capas con responsabilidades separadas. La API se organiza en controladores, servicios y repositorios.

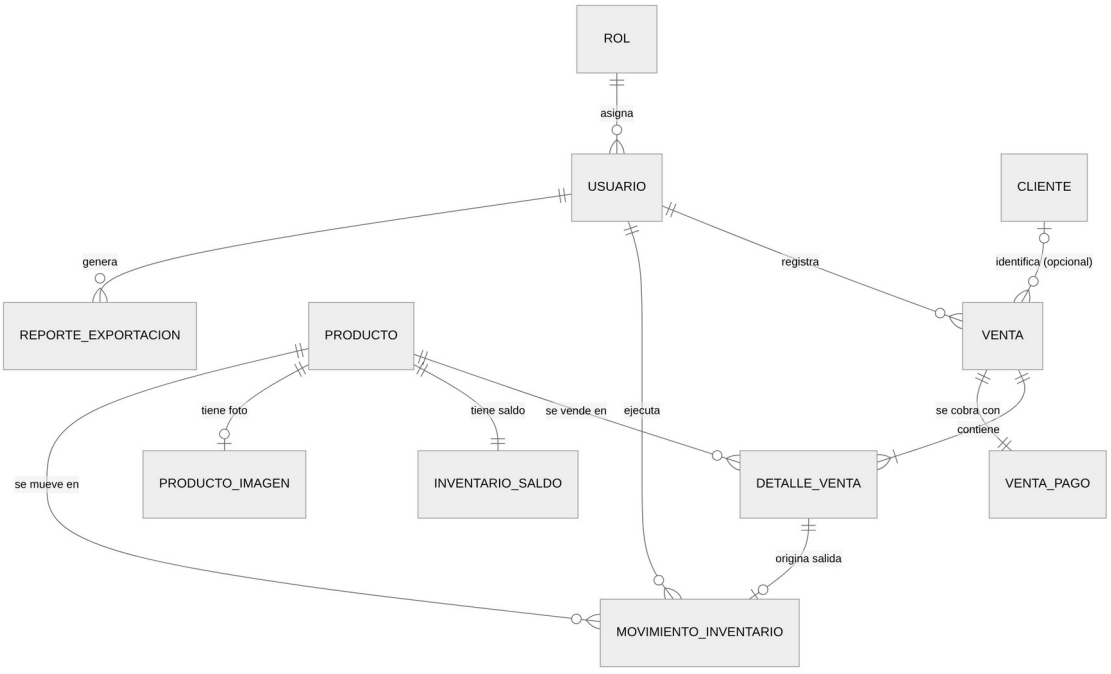
Capa	Tecnología	Responsabilidad
App cliente	React 19 + Capacitor 8 (Android)	Interfaz, formularios, carrito local (borrador) y solicitudes HTTP con token JWT.
Controladores	FastAPI · app/controladores	Rutas, códigos HTTP, autenticación y validación de entrada (Pydantic).
Servicios	Python · app/servicios	Reglas de negocio, permisos por rol, transacciones e idempotencia.
Repositorios	psycopg 3 · app/repositorios	SQL parametrizado; sin lógica de negocio.
Base de datos	PostgreSQL 16 en local · 18 en Neon	Integridad con PK, FK y CHECK; triggers de stock e inmutabilidad; vistas para reportes.

Qué hace la API y no el cliente. Decide los precios desde el catálogo, calcula el total y el vuelto, verifica el stock, aplica los permisos según el rol leído de la base de datos y registra la venta en una sola transacción. La app solo envía qué productos, cuántos y cómo se paga.

La API está desplegada en Vercel (`https://bodegacontrol.vercel.app`) y la base de datos en Neon, ambas en la región de São Paulo. La app se conecta por HTTPS a esa dirección, que también puede cambiarse en la pantalla de inicio de sesión.

3. Modelo de datos

El esquema tiene 11 tablas relacionadas, 6 vistas y 10 funciones con triggers. El diagrama muestra las entidades y sus relaciones; el detalle de cada atributo está en base-datos/DICCIONARIO_DE_DATOS.md.



Decisión de diseño	Motivo
Stock = suma de movimientos	inventario_saldo solo cambia por un trigger al insertar en movimiento_inventario (ALTA, ENTRADA, SALIDA_VENTA, MERMA). Una actualización directa del saldo se rechaza.
Precio y costo históricos	detalle_venta guarda el precio y el costo del momento. Un cambio posterior de precio no altera ventas pasadas.
Registros inmutables	Ventas, detalles, pagos y movimientos no se editan ni se borran. Los productos se desactivan (baja lógica).
Correlativos en la base	El código de producto (P0001) y el N.º de entrada (E-00001) los asignan secuencias de PostgreSQL.

4. Endpoints

La API expone 24 operaciones bajo `/api/v1`. La app Android consume las 24. La documentación completa, con parámetros y ejemplos JSON reales, está en `docs/API.md`, en Swagger (`/docs`) y en la colección Postman de `docs/postman`.

Método	Ruta	Rol	Códigos HTTP
POST	/auth/login	Público	200 · 401
GET	/auth/sesion	Ambos	200 · 401
GET	/productos	Ambos	200
POST	/productos	Propietario	201 · 400 · 403 · 409
GET	/productos/{id}	Ambos	200 · 404
GET	/productos/codigo-barras/{codigo}	Ambos	200 · 404
PATCH	/productos/{id}	Propietario	200 · 400 · 404 · 409
DELETE	/productos/{id}	Propietario	204 · 404
PUT · GET · DELETE	/productos/{id}/imagen	Propietario (GET público)	200 · 204 · 413 · 422
GET	/inventario/{id}	Ambos	200 · 404
POST	/inventario/{id}/entradas	Propietario	201 · 400 · 422
POST	/inventario/{id}/mermas	Propietario	201 · 400 · 409
GET · POST	/clientes	Ambos	200 · 201 · 400 · 409
POST	/ventas	Ambos	201 · 200 · 400 · 409 · 422
GET	/ventas/estado?clave=	Ambos	200 · 202 · 404
GET	/ventas · /ventas/{id}	Ambos	200 · 404
GET	/reportes/resumen	Propietario	200 · 403
GET	/reportes/{ventas|mermas}	Propietario	200 · 400 · 403
GET	/reportes/exportaciones	Propietario	200 · 403
GET	/salud	Público	200 · 503

Operación de negocio: confirmar venta

`POST /api/v1/ventas` recibe la clave de idempotencia, las líneas y el pago. No recibe precios ni totales.

```
{ "clave_idempotencia": "8e710ad8-92b2-4216-a290-bdcbda178fb4", "cliente_id": 5,
  "lineas": [{ "producto_id": 1, "cantidad": 2 }, { "producto_id": 4, "cantidad": 1 }],
  "pago": { "metodo": "EFECTIVO", "importe_recibido": 20.00 } }
```

Respuesta real (201): `"venta_id": 20, "total": 11.8, "vuelto": 8.2, "repetida": false`, con el detalle de cada línea y el movimiento de inventario que generó.

5. Reglas

Regla del servidor	Cómo se garantiza
Venta completa o nada	Venta, detalle, pago y movimientos se insertan en una transacción. Si una regla falla, se revierte todo. Un trigger diferido rechaza ventas sin detalle o sin pago.
Sin stock negativo	Los saldos se bloquean con FOR UPDATE en orden de producto_id. Un CHECK impide saldo menor que cero aunque falle el backend.
Sin ventas duplicadas	La app genera una clave UUID por venta. Un reintento con la misma clave devuelve la venta existente (200). La misma clave con otro contenido devuelve 409.
Reconciliación tras corte de red	Ante un timeout, la app consulta /ventas/estado antes de reenviar. No anuncia una venta como confirmada sin respuesta del servidor.
Precios del servidor	El total se calcula con el precio vigente del catálogo. Un precio enviado por la app se rechaza.
Pagos verificados	Yape, Plin y tarjeta exigen N.º de operación único. El efectivo exige importe recibido mayor o igual al total.
Permisos por rol	El rol se lee de la base de datos en cada solicitud. El cajero no ve costos ni ganancias y solo ve su historial.
Mermas trazables	Cada merma registra motivo y costo unitario congelado; la pérdida se calcula en soles.

6. Pruebas

Las pruebas se ejecutan sobre PostgreSQL real y comprueban la fila guardada con una consulta SQL directa, no solo la respuesta HTTP.

Caso	Prueba	Resultado esperado	Resultado
T-01	Crear, listar, editar y desactivar producto	201 · 200 · 204 y cambios en la tabla producto	Correcto
T-02	Código de barras duplicado	409 y ninguna fila adicional	Correcto
T-03	Venta válida en efectivo	201; venta, detalle, pago, movimientos y stock coherentes	Correcto
T-04	Venta con stock insuficiente	409 con la línea afectada; ninguna escritura	Correcto
T-05	Reintento con la misma clave	Una sola venta; otro contenido con la misma clave → 409	Correcto
T-06	Corte de red y consulta por clave	404 antes de registrar; 200 después	Correcto
T-07	Cajero exporta reportes o crea productos	403 y sin registro de auditoría	Correcto
T-08	Reporte PDF y CSV por periodo	Total igual a la suma en SQL; periodos en hora de Lima	Correcto
T-09	Pago Yape sin referencia o repetida	400 y 409	Correcto

Suite	Casos	Resultado
Integridad del esquema (base-datos/03_pruebas_esquema.py)	47	47 correctos
API con pytest y PostgreSQL (backend/pruebas)	47	47 correctos
Colección Postman con Newman (docs/postman)	49 solicitudes · 68 verificaciones	Sin fallos
App de extremo a extremo con Playwright (app-android/pruebas-e2e)	21 flujos	21 correctos